Exception Handling

02 April 2026 20:24

Exception handling is one of the most powerful mechanism to handle runtime errors so that normal flow of the application can be maintained.

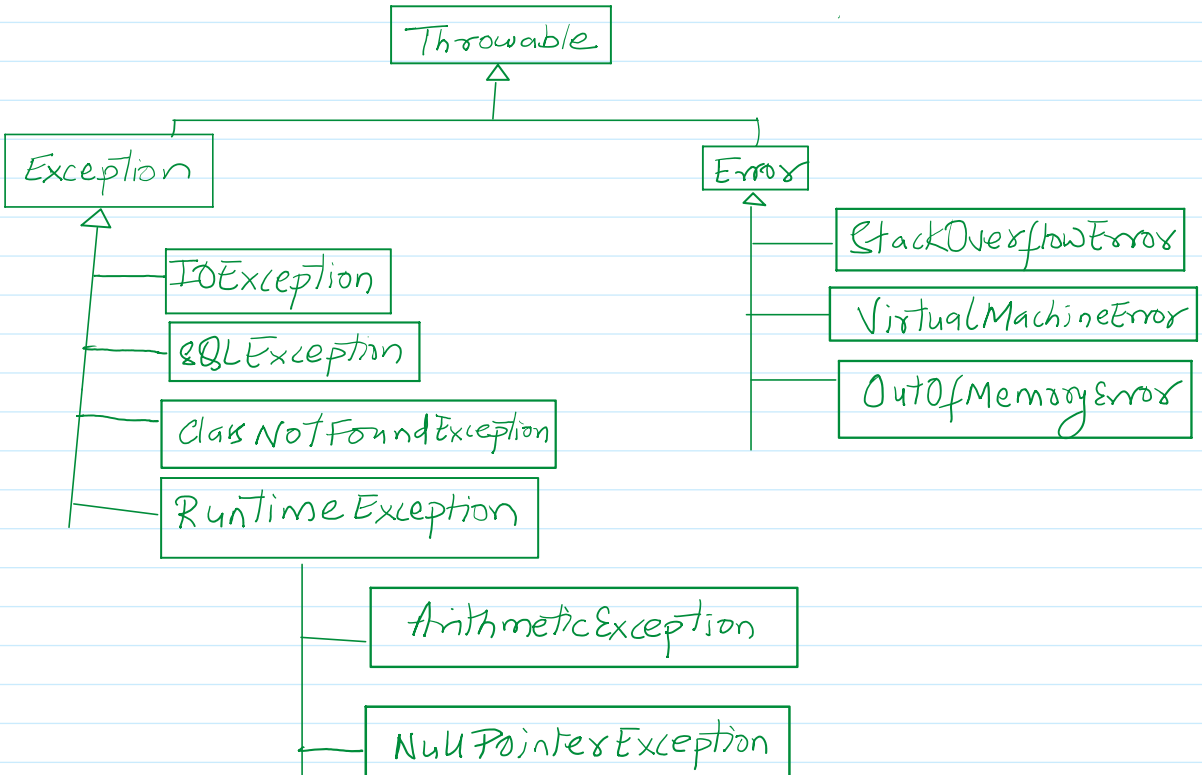
What is Exception in Java?
In Java, an exception is an event that occurs during the execution of a program and disrupt the normal flow of instructions. Exception can occur for various reasons such as invalid user input of file not being found or division by zero error. When an exception occurs, it is typically represented by an object of a subclass of the **java.lang.Exception** class.

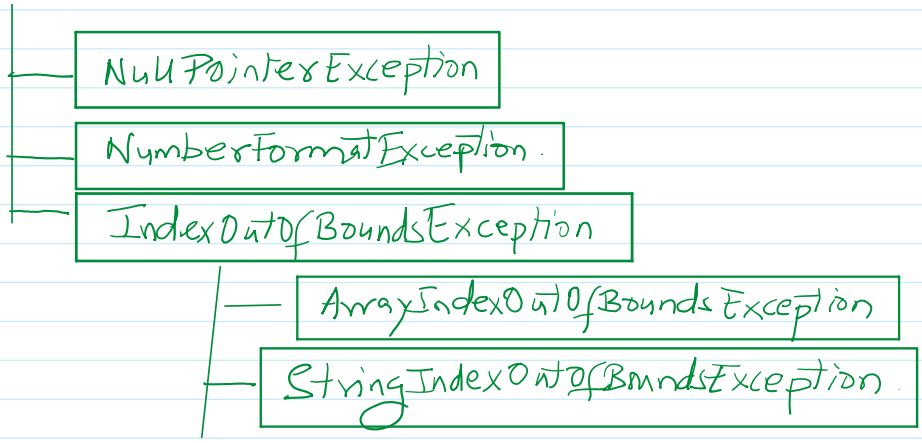
What is Exception Handling in Java?
Exception Handling is a mechanism that allow a program to handle **runtime errors** such as ClassNotFoundException, IOException, SQLException, InputmismatchException, and others, so that normal flow of the application can be maintained.

Hierarchy of Java Exception classes
Exceptions are represented by classes and all exception classes are the part of java.lang. package. They are organized in a hierarchical structure with Throwable at the top. The main hierarchy is as follows:

- **Throwable:** The super class of all errors and exceptions in Java.
  - **Error:** Represents serious system errors that application usually cannot handle (e.g., OutOfMemoryError).
  - **Exception:** Represents conditions that a program might want to catch.
    - **Checked Exception(Compile-time Exception): Must be either handled using try-catch or declared using throws(e.g., IOException, and many more)**

    - **Unchecked Exception (Runtime Exception): Do not need explicit handling, typically caused by programming errors(e.g., NullPointerException, ArithmeticException, and many more)**





Types of Java Exception

The exceptions are categorized in two main types: checked exceptions and unchecked exceptions. Additionally, there is a 3rd category known as Errors.

- **Checked Exception:** Exception that must be handled or declared in the code.
- **Unchecked Exception:** Exceptions that do not require explicit handling and usually occur due to programming errors.
- **Error: Serious problems that are not meant to be caught by applications, usually system-related.**

```
package ExceptionHandling;


/**
 * Write a description of class CheckedExceptionExample here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class CheckedExceptionExample
{
    public static void main(String[] args) throws java.io.IOException {
        char c;
        String v="aeiouAEIOU";
        System.out.print("\fEnter any character: ");
        c = (char)System.in.read();
        if(Character.isAlphabetic(c)){
            System.out.print("\n" + c + " is an alphabet");
            if(Character.isUpperCase(c)){
                System.out.print("\n" + c + " is an uppercase alphabet.");
            }
            else{
                System.out.print("\n" + c + " is an lowercase alphabet.");
            }
            if(v.indexOf(c) > -1){
                System.out.print("\n" + c + " is a vowel.");
            }
            else{
                System.out.print("\n" + c + " is not a vowel");
            }
```

```
        }
        else if(Character.isWhitespace(c)){
            System.out.print("\n it is a white space." );
        }
        else if(Character.isDigit(c)){
            System.out.print("\n" + c + " is a digit.");
        }
        else{
            System.out.print("\n" + c + " is an other character.");
        }

    }
}
```

Outputs:

```
BlueJ: Terminal Window - AaditClass7
Options
Enter any character: @

@ is an other character.
```

```
BlueJ: Terminal Window - AaditClass7
Options
Enter any character: 4

4 is a digit.
```

```
BlueJ: Terminal Window - AaditClass7
Options
Enter any character:

 it is a white space.
```

```
BlueJ: Terminal Window - AaditClass7
Options
Enter any character: q

q is an alphabet
q is an lowercase alphabet.
q is not a vowel
```

```
BlueJ: Terminal Window - AaditClass7
Options
Enter any character: e

e is an alphabet
e is an lowercase alphabet.
e is a vowel.
```

Java unchecked exceptions(runtime exceptions)

Unchecked exceptions, also known as runtime exceptions, are not checked at compile-time. These exceptions usually occur due to programming errors such as logic errors or incorrect assumption in the code. They do not need to be declared in the method signatures using the throw keyword, making it optional to handle them. Examples of a

checked unchecked exceptions include:

- **NullPointerException:** It is thrown when trying to access or call a method on an object reference that is null.

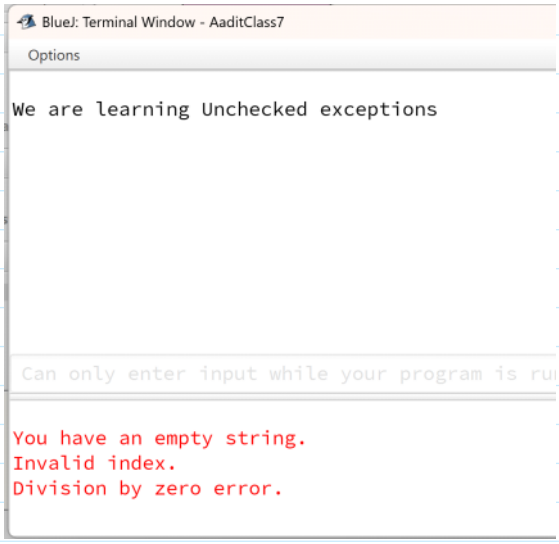
- **ArrayIndexOutOfBoundsException:** It occurs when we try to access an array element with an invalid index.

- **ArithmeticException:** It is thrown when an arithmetic operation fails, such as division by 0.

- **IllegalArgumentException:** It indicates that a method has been passed an illegal or inappropriate argument.

```
package ExceptionHandling;

/**
 * Write a description of class UncheckedExceptionExample here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class UncheckedExceptionExample
{
    public static void main(String[] args){
        String str = null;
        System.out.print("\f");
        try{
            System.out.print("Length of str: "+ + str.length());
        }
        catch(NullPointerException npe){
            System.err.print("\nYou have an empty string.");
        }

        int[] arr = {1,2,3};
        try{
            System.out.print("\n"+arr[5]);
        }
        catch(ArrayIndexOutOfBoundsException aiobe){
            System.err.print("\nInvalid index.");
        }

        try{
            int r = 10/0;
        }
        catch(ArithmeticException ae){
            System.err.print("\nDivision by zero error.");
        }
        System.out.print("\nWe are learning Unchecked exceptions");
    }
}
```

Output:



Java Exception Handling Keywords

Java provides five keywords that are used to handle the exception. The following table describes each.

Keyword	Description
try	The **"try"** keyword is used to specify a block where we should place an exception code. It means we can't use try block alone. The **try** block must be followed either catch or finally block.
catch	The **"catch"** block is used to handle the exception. It must be preceded by **"try"** block which means we can't use **"catch"** block alone. It can be followed by **"finally"** block later.
finally	The **"finally"** block is used to execute the necessary code of the program. It is executed whether an exception is handled or not.
throw	The throw keyword is used to throw an exception.
throws	The **"throws"** keyword is used to declare exceptions. It specifies that there may occur an exception in the method. It does not throw an exception. It is always used with the method signature.

The try-catch Block

The try - catch block is a primary mechanism for handling exceptions. The tribe lock contains code that may throw an exception, and the catch block handles that exception.

Syntax:

```
    try{
      // Code that may throw an exception.
    }
    catch(ExceptionType e){
      // Code to handle the exception.
    }
   package ExceptionHandling;

   /**
    * Write a description of class UncheckedExceptionExample here.
```

```
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class UncheckedExceptionExample
{
    public static void main(String[] args){
        String str = null;
        System.out.print("\f");
        try{
            System.out.print("Length of str: "+ + str.length());
        }
        catch(NullPointerException npe){
            System.err.print("\nYou have an empty string.");
        }

        int[] arr = {1,2,3};
        try{
            System.out.print("\n"+arr[5]);
        }
        catch(ArrayIndexOutOfBoundsException aiobe){
            System.err.print("\nInvalid index.");
        }

        try{
            int r = 10/0;
        }
        catch(ArithmeticException ae){
            System.err.print("\nDivision by zero error.");
        }
        System.out.print("\nWe are learning Unchecked exceptions");
    }
}
```

Homework for practice:

1. Write a Java program to create a student result management system. The program should accept marks of multiple subjects, calculate the percentage, and determine the grade. Handle exceptions such as invalid marks (less than 0 or greater than 100), division by zero while calculating percentage, and null student names. Use try-catch-finally blocks and create a custom exception called InvalidMarksException.

2. Write a Java program for a bank account system that supports deposit, withdrawal, and balance checking. Throw an exception when the withdrawal amount exceeds the available balance. Also handle invalid amounts such as negative deposits or withdrawals. Use throw, throws, multiple catch blocks, and a custom exception named InsufficientBalanceException.

3. Write a Java program to read data from a text file and display its contents line by line. Handle checked exceptions such as FileNotFoundException and IOException. Ensure that the file stream is closed properly using a finally block.

4. Write a Java program that takes an array of integers from the user and performs operations such as finding the largest number, smallest number, and average. Handle

exceptions such as ArrayIndexOutOfBoundsException, InputMismatchException, and ArithmeticException.

5. Write a Java program to simulate an online shopping cart. The program should allow the user to add products, remove products, and calculate the total price. Handle exceptions for invalid product quantity, negative price, and null product names. Create a custom exception named InvalidProductException.

6. Write a Java program that demonstrates nested try-catch blocks. The program should:
   - Accept two numbers and divide them
   - Store the result in an array
   - Access an array element using a user-provided index
     Handle ArithmeticException, ArrayIndexOutOfBoundsException, and InputMismatchException.

7. Write a Java program for a railway reservation system. The program should allow booking tickets for passengers. Handle exceptions when:
   - The passenger enters an invalid age
   - The requested number of seats exceeds available seats
   - The train number is invalid
     Use custom exceptions and demonstrate the use of throw and throws.

8. Write a Java program to demonstrate the use of throws keyword. Create methods for:
   - Reading a file
   - Parsing an integer from a string
   - Dividing two numbers
     Declare exceptions using throws and handle them in the main() method.

9. Write a Java program for employee salary processing. The program should calculate salary based on basic pay, bonus, and deductions. Handle exceptions for invalid salary values, null employee names, and invalid employee IDs. Use multiple catch blocks and a custom exception named InvalidEmployeeDataException.

10. Write a Java program to simulate a library management system. The system should allow issuing and returning books. Handle exceptions when:
    - A book is not available
    - The user enters an invalid book ID
    - The member ID is incorrect
    - The return date exceeds the due date

    Use custom exceptions, try-catch-finally, and proper exception messages.

Handling Multiple Exceptions

Java allows handling different types of exception using multiple cache blocks. Each catch block is used for a specific exception type.

Syntax:

```
try{
    //Code that may throw an exception
}
catch(IOException e){
  //Code to Handle IOException
```

```
    }
    catch(NumberFormatException e){
      //Code to handle NumberFormatException
    }
    catch(Exception e){
       //Handle any other exception
    }
```

Example of multiple catch block:

```
package ExceptionHandling;
import java.util.*;

/**
 * Write a description of class MultipleException here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class MultipleException
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String name = null;
        int num[] = {1,0,3};
        try{
            //System.out.print("\fnum[10]: "+num[10]);
            System.out.print("\fnum[2]: "+num[2]);
            System.out.print("\nEnter any value in num[2]: ");
            num[2] = sc.nextInt();
            name = "Joy";
            System.out.print("\nname =" + name.length());
            System.out.print("\nQuotient by dividing " + num[2] + "/" + num[1] +
            " = " + num[2]/num[1]);
        }
        catch(ArrayIndexOutOfBoundsException e){
            System.err.print("\nInvalid index value.\n ");
            //e.printStackTrace();
        }
        catch(InputMismatchException e){
            System.err.print("\nYou have not entered integer value.");
        }
        catch(NullPointerException e){
            System.err.print("\nYou have not entered any data in String variable");
        }
        catch(ArithmeticException e){
            System.err.print("\n Divide by zero error!");
        }
    }
}
```

The finally Block

The finally block is executed regardless of whether an exception occurs or not. It is commonly used to release resources like files or database connections.

Syntax:

```
try{
   //Code that may throw an exception
}
catch(Exception e){
   //Exception handling code
}
finally{
   //cleanup code
}
```

```
package ExceptionHandling;


/**
 * Write a description of class TryCatchFinally here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class TryCatchFinally
{
    public static void main(String[] args){
        System.out.print("\f");
        try{
            int data = 25/0; //May throw ArithmeticException
            System.out.print("\ndata = " + data);
        }
        catch(ArithmeticException e){
            System.err.print("\nError: Division by zero not allowed.");
        }
        finally{
            System.out.print("\nI am the finally block, and I always executes");
        }
        System.out.print("\nProgram continues after finally block...");
    }
}
```

Output:

Revise:

Java try-catch block: The try block is used to enclose the code that might throw an exception during execution. Any code that has the potential to cause an error, such as division by zero, accessing an invalid array index, or reading a file, should be placed inside the try block.

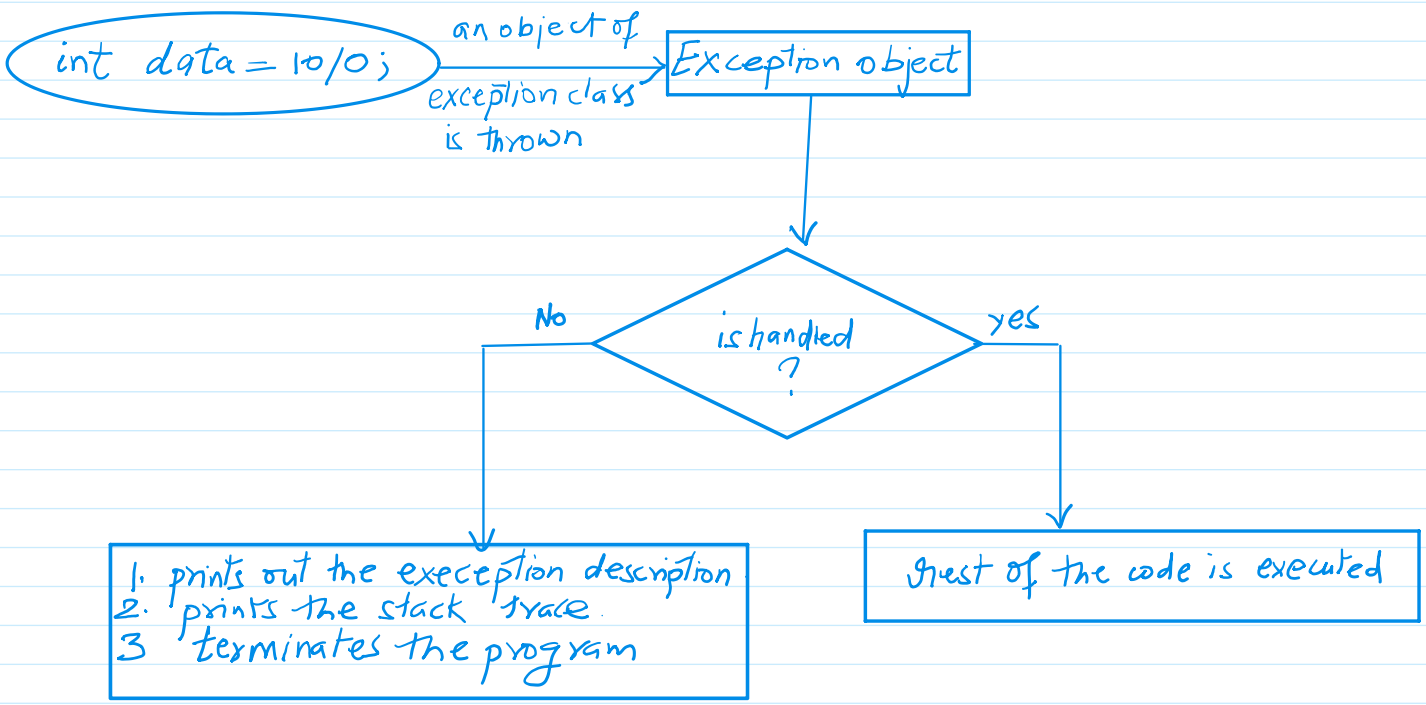
The catch block is used to handle the exception thrown by the try block. You specify the type of exception to catch inside parenthesis. When an exception occurs in the try block, the corresponding catch block executes.

- The try block contains the code that might cause an exception.
- The catch block handles the exception if it occurs.
- Exception type can be a specific exception like ArithmeticException, IOException or the general Exception class.
- The catch block receives an exception object (commonly named E) That contains details about the exception.

How Java Handles Exceptions using try-catch block?
When an exception occurs, the Java virtual machine (JVM) First checks whether the exception is handled by the program.

- **If the exception is not handled:**
  - The JVM provides a default exception handler that performs the following task:
    - Prints the exception description.
    - Prints those stack trees (the hierarchy of methods where the exception occurred)
    - Terminates the program.
- **If the exception is handled by the programmer:**
    - The normal flow of the application is maintained and rest of the code continues to execute.

This mechanism makes sure that exception do not always crash the program and gives programmer a way to recover from runtime errors gracefully.



Nested try-catch block

In Java, using a try block inside another try block is permitted. It is called a nested try block. Every statement that we enter a statement in try block context of that exception is pushed onto the stack.

For example, the inner try block can be used to handle ArrayIndexOutOfBoundException, while the outer try block can handle the ArithmeticException (division by zero).

Why use nested try Block?

Sometimes a part of a block may cause one type of exception, while the entire block itself may cause another. In such cases, nested try blocks are useful to handle multiple exceptions separately.

Syntax of Nested try-catch

A nested try block is a Tri block placed inside another Tri block. The outer try block can handle exception not caught by the inner try block(s).

```
try{
   //Outer try block
   //code that may throw an exception

   try{
      //inner try block
      //code that may throw an exception
   }
   catch([ExceptionType] e1){
      // Inner catch block handles ExceptionType.
   }
   // Optional more inner try-catch block can be added.
}
catch(ExceptionType2 e){
   // Powder catch block handles exception not caught by inner try block.
}
```

* Outer try block contains the main code that may throw exceptions.
* If an exception occurs that is not handled by any inner try-catch, it is passed to this outer block.
→ The inner try block(s) is/are placed inside the outer try-block.
* These can handle exception specific to the inner code.
* Each try block (inner or outer) must have at least one corresponding catch block to handle exceptions.
* The outer catch block act as a backup handler for any exception not handled by inner catch blocks.

For example:

```
import java.util.*;

public class ArrayAccess{
public static void main(String args[]){
        Scanner sc = new Scanner(System.in);
        System.out.print("\f");
        double arr[] = new double[6];
        for(int i=0;i<6;i++){
            try{
                System.out.print("\nEnter the Numerator : ");
                int n = sc.nextInt();
                System.out.print("\nEnter the Denominator : ");
                int d = sc.nextInt();
                try{ //nested try
                    int r = n/d;
                    arr[i] = r;
                }
                catch(ArithmeticException ae){ //nested catch
```

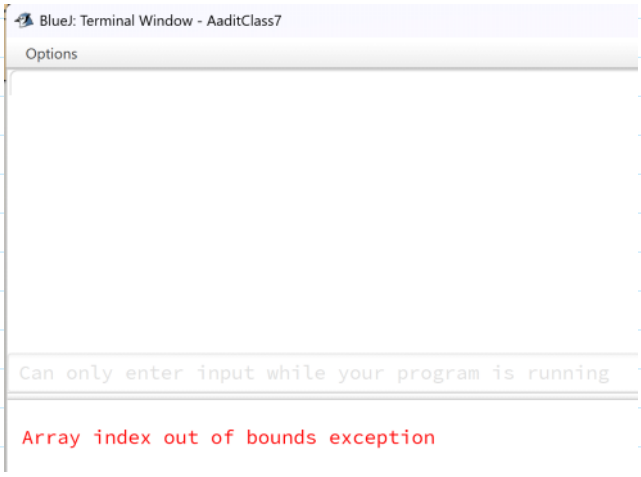
```
                System.err.print("\nDividing by 0 is incorrect ");
            }
            }
            catch(InputMismatchException ime){ //outer-catch
                System.err.print("\nInput should be in numerical format ");
                i--;
                sc.nextLine();
            }
        }
        System.out.print("\n\n[");
        for(int j=0;j<6;j++){
            System.out.print(arr[j] + " ");
        }
        System.out.print("]");
        try{
            System.out.print("\nEnter Index Of Array Element(1-6) : ");
            sc.nextLine();
            int ind = sc.nextInt();
            System.out.print("\nArray Element : " + arr[ind-1]);
        }
        catch(ArrayIndexOutOfBoundsException aie){
            System.err.print("\nIncorrect Index has been entered ");
        }
    }
```

Another example:

```
package ExceptionHandling;
import java.util.InputMismatchException;


/**
 * Write a description of class NestedTryBlockOne here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class NestedTryBlockOne
{
    public static void main(String[] args){
        try{
            try{
                try{
                    int arr[] = {1, 2, 3, 4};
                    System.out.print("\n"+arr[10]);
                }
                catch(ArithmeticException e){
                    System.err.print("\nArithmetic exception");
                }
            }
            catch(InputMismatchException e){
                System.err.print("\nInput mismatch exception");
            }
        }
        catch(ArrayIndexOutOfBoundsException e){
```

```
            System.err.print("\n Array index out of bounds exception");
        }
        catch(Exception e){
            System.err.print("\nException");
            System.err.print("\nHandled the main try-block");
        }
    }
}
```

Output:

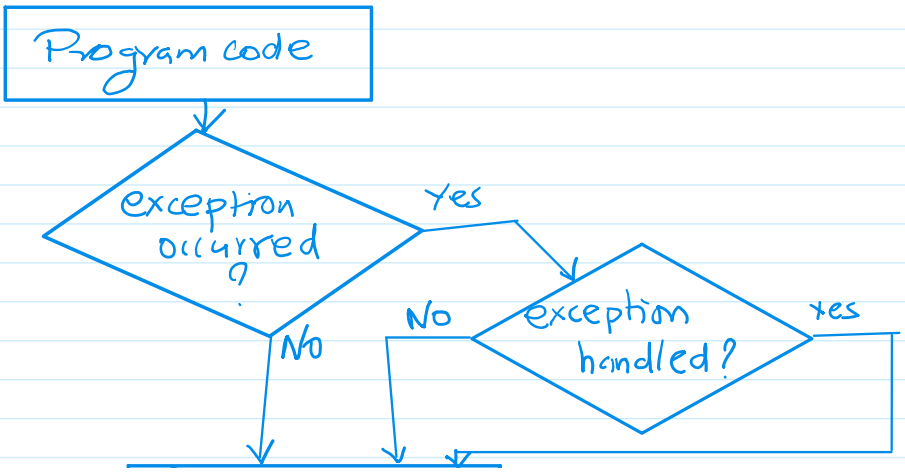


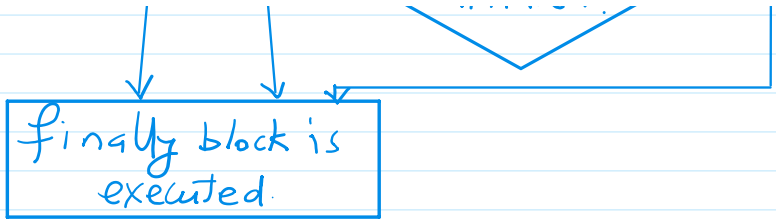
finally Block in Java

In Java, the finally block is used to execute important codes such as closing files or releasing resources regardless of whether an exception occurs or not.

What is the finally Block in Java?
The finally block is a block of code that always executes after a try block, whether an exception thrown or not. The finally block is useful when you want to clean up activities or free the resource, like closing files, streams or database connections.

Key points
1. A **finally block** is optional, but when present it always executes after the try and catch blocks.
2. It executes even if a return statement is encountered in the try or catch blocks.

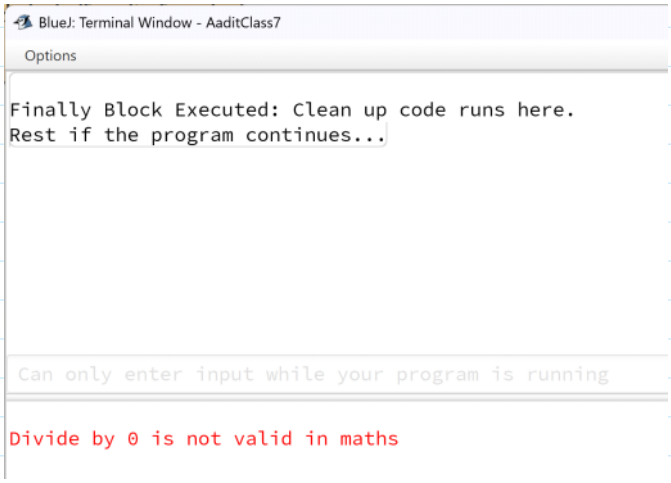




For example:

```
package ExceptionHandling;


/**
 * Write a description of class FinallyBlockone here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class FinallyBlockOne
{
    public static void main(String[] args){
        try{
            int data = 100/0;
        }catch(ArithmeticException e){
            System.err.print("\nDivide by 0 is not valid in maths");
        }finally{
            System.out.print("\nFinally Block Executed: Clean up code runs
here.");
        }
        System.out.print("\nRest if the program continues...");
    }
}
```

Output:

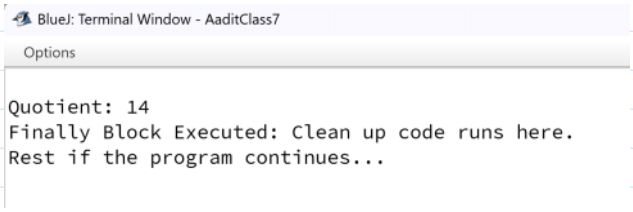


Another example, where catch block is not executed:

```
package ExceptionHandling;


/**
```

```
 * Write a description of class FinallyBlockone here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class FinallyBlockOne
{
    public static void main(String[] args){
        System.out.print("\f");
        try{
            int data = 100/7;
            System.out.print("\nQuotient: " + data);
        }catch(ArithmeticException e){
            System.err.print("\nDivide by 0 is not valid in maths");
        }finally{
            System.out.print("\nFinally Block Executed: Clean up code runs here.");
        }
        System.out.print("\nRest if the program continues...");
    }
}
```

Output:



Java Throw Exception

In Java, the throw keyword is used to explicitly throw an exception within a method or block of code. It allows developers to signal that an error has occurred and needs to be handled. The throne exception must be an instance of Throwable or one of its subclasses, such as Exception or RuntimeException.

Purpose of throw Keyword

The Java throw keyword is used to throw an exception explicitly from a method or a block of code. It can be used to throw both checked and unchecked exceptions. It is mainly used to throw a custom exception and allows developer to manually trigger an exception when an undesirable situation occurs.

Exception Object and Error Message

When using the throws keyword, we specify the exception object that needs to be thrown. The exception object contains a message that describes the error. These exceptions may be related to user inputs, serve issues, hardware malfunctions, invalid computations, or other conditions based on the programs requirements.

Rules for Throwing Exceptions

The following rules must be followed during throwing exceptions:

- The object that is thrown must be of type Throwable or one of its subclasses.
- The Exception class is a direct subclass of Throwable.
- User defined exceptions, also known as custom exceptions, usually extend the Exception.

- If a checked exception is thrown and not handled within the method, it must be declared using the throws keyword in the method signature.

Throwing an Exception using "throw" keyword

The throw keyword is used to explicitly throw an exception from a method or a block of code. It allows the programmer to manually trigger an exception when a specific condition occurs that disrupt the normal flow of the program.

Syntax:

throw new Exception_class("error message");

Where instance refers to an object of a class that must extend Throwable(Which includes the exception class and its subclasses).

Example:

throw new IOException("Sorry device error");

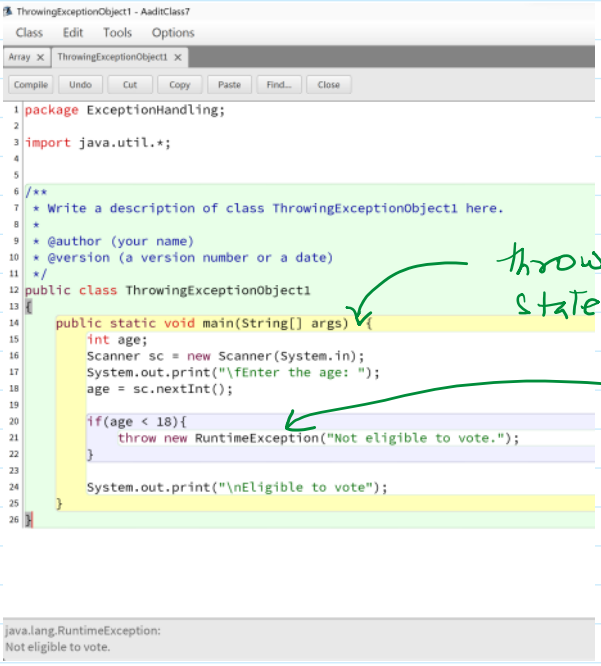
Here:

- **new** is used to create an instance (object) of the Exception class.
- The error message proof. Provides the context about the reason for the exception.

Where the instance must be of type Throwable or a subclass of Throwable. For example, Exception is a subclass of Throwable and the user defined Exception usually extends the Exception class.

```java
package ExceptionHandling;
import java.util.*;


/**
 * Write a description of class ThrowingExceptionObject1 here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ThrowingExceptionObject1
{
    public static void main(String[] args){
        int age;
        Scanner sc = new Scanner(System.in);
        System.out.print("\fEnter the age: ");
        age = sc.nextInt();

        if(age < 18){
            throw new ArithmeticException("Not eligible to vote.");
        }

        System.out.print("\nEligible to vote");
    }
}
```

```
java.lang.ArithmeticException:
Not eligible to vote.
```



throws clause is not needed, when throw statement is throwing Unchecked Exceptions.

this must be unchecked exception.

Throwing an Unchecked Exception

And unchecked exception is an exception that is not checked at compile time and usually occurs due to programming errors such as invalid calculations or improper use of objects. In Java, unchecked exceptions extend the RuntimeException class and can be thrown using the throw keyword without declaring them using the throws clause.

Throwing a Checked Exception

For checked exception is an exception that is checked at compile time and must be either handled using a Tri - catch block or declared using the throws keyword in the method signature. In Java checked exception extend the Exception class (excluding RuntimeException).

```
package ExceptionHandling;
import java.io.*;


/**
 * Write a description of class ThrowingCheckedException1 here.
```

```
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ThrowingCheckedException1
{
    public static void method() throws IOException {
        int x;
        InputStreamReader isr = new InputStreamReader(System.in);
        BufferedReader br = new BufferedReader(isr);

        System.out.print("\fEnter the value of x: ");
        x = Integer.parseInt(br.readLine());
        throw new IOException("Data entered in String format, converted into an int
format");
    }

    public static void main(String[] args){
        try
        {
            method();
        }
        catch (IOException ioe)
        {
            ioe.printStackTrace();
        }
    }
}
```

Output:

Throwing a Checked Exception

A checked exception is an exception that is checked at compile time and must be either handled using a try-catch block or declared using the **throws** keyword in the method signature. In Java check the exception extend the Exception class (excluding RuntimeException)

```
package ExceptionHandling;
```

```
/**
 * Write a description of class CheckedException here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class CheckedException
{
    public static void main(String[] args) throws InterruptedException {
        int i;
        System.out.print("\f");
        for(i = 0; i < 10; i++){
            System.out.print("\nSecond: " + i);
            Thread.sleep(1000);
        }
        System.out.print("\nSecond: " + i);
    }
}
```

Throwing User-defined Exception

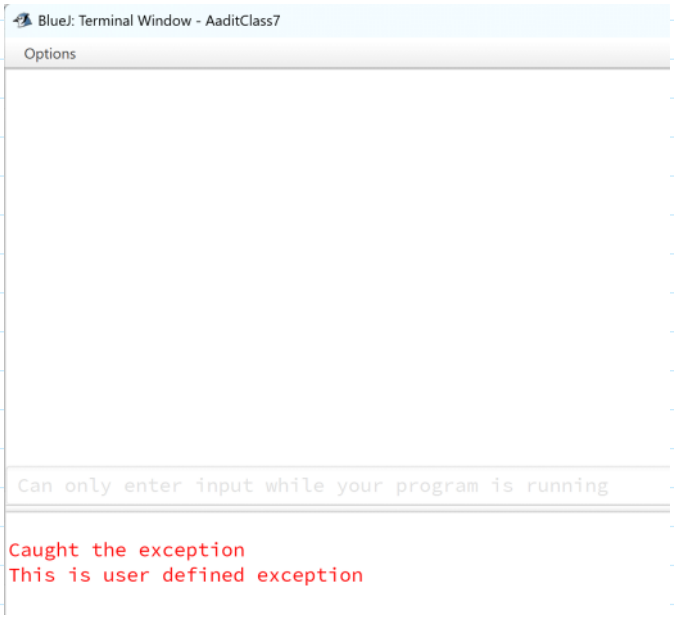
The exception is everything else under the Throwable class. Java allows developer. To create their own custom exceptions by extending the exception class or one of its subclasses. These are known as User defined exceptions. They are typically used when you want to enforce application specific rules and constraints that are not covered by Java built-in exceptions.

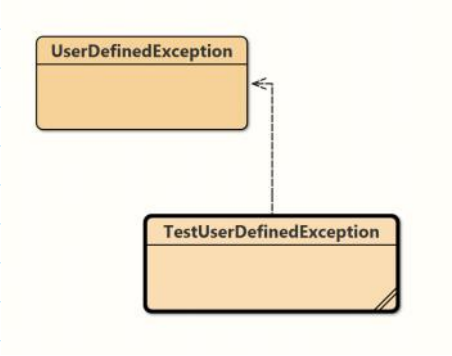
Example: In this example we will define a custom exception class called UserDefinedException. And throw it manually using the throw keyword. We will also catch and handle it using a try cache block just like we would with any other checked exception.

```
public class UserDefinedException extends Exception
{
    public UserDefinedException(String msg){
        super(msg);
    }
}


public class TestUserDefinedException
{
    public static void main(String[] args){
        System.out.print("\f");
        try
        {
            throw new UserDefinedException("This is user defined exception");
        }
        catch (ExceptionHandling.UserDefinedException ude)
        {
            System.err.print("\nCaught the exception");
            System.err.print("\n" + ude.getMessage());
        }
```

```
    }
}
```

Output:





Class relationship diagram

Exception Propagation in Java
Exception propagation in Java is used for allowing exceptions to move up the call stack so they can be handled at the higher level in the program.

What is Exception Propagation in Java?
Exception propagation is a mechanism that allows an exception to be passed from a method where it occurs to a higher level methods in the call stack. Exception propagation gives the flexibility to the programmers to decide where to catch and handle the exception.

When an exception is thrown, it first looks for a matching catch block in the current method. If not found, the exception propagates down the call stack to the previous method, continuing this process until it is caught or reaches the bottom of the stack. The process of moving an exception through the calling chain is why it is called exception propagation.

Note: By default, unchecked exception (subclass of RuntimeException) are automatically propagated up the calling chain.

How does exception propagation works?
Exception propagation occurs when an exception is thrown from a method and is not caught within that method. The Java runtime system will look. For a handler in the calling method. The method that is called the current method. If the calling method does not handle the exception, it gets propagated further up the call stack. The continues until either:

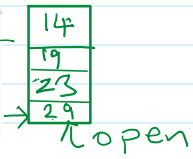
- A method catches the exception (That is a matching catch block is found).
- The exception reaches to the main () method which serves as the entry point to the program.

If no method handles the exception, the program terminates and displays the stack

trace, which helps to identify the origin and path of the exception.

```java
10 public class ExceptionPropagation
11 {
12     public void m(){
13         int a = 100;
14         int b = a / 0; //   Divide by zero throws ArithmeticException.
15
16     }
17
18     public void n(){
19         m(); // Exception is not handled here.
20     }
21
22     public void p(){
23         n(); // Exception propagates to here.
24     }
25
26     public static void main(String[] args){
27         System.out.print("\f");
28         ExceptionPropagation obj = new ExceptionPropagation();
29         obj.p(); //   Starts the chain of method call.
30         // The line is executed only if the exception is handled properly.
31         System.out.print("\nNormal flow...");
32     }
33 }
```

```
java.lang.ArithmeticException: / by zero
        at ExceptionHandling.ExceptionPropagation.m(ExceptionPropagation.java:14)
        at ExceptionHandling.ExceptionPropagation.n(ExceptionPropagation.java:19)
        at ExceptionHandling.ExceptionPropagation.p(ExceptionPropagation.java:23)
        at ExceptionHandling.ExceptionPropagation.main(ExceptionPropagation.java:29)
```



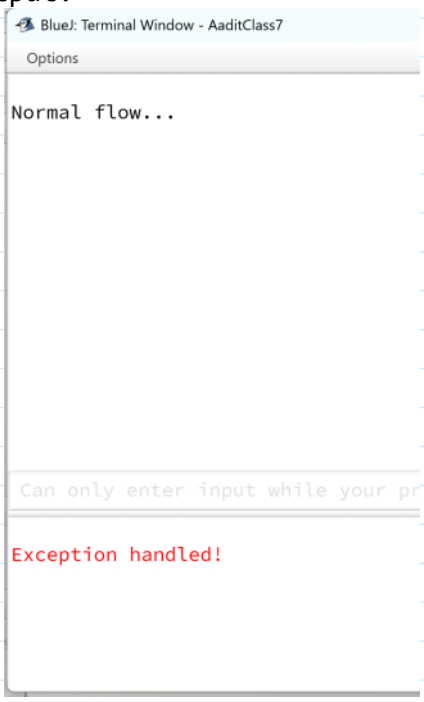

```java
package ExceptionHandling;


/**
 * Write a description of class ExceptionPropagation here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ExceptionPropagation
{
    public void m(){
        int a = 100;
        int b = a / 0; //   Divide by zero throws ArithmeticException.

    }

    public void n(){
        m(); // Exception is not handled here.
    }

    public void p(){
        try{
            n(); // Exception propagates to here.
        }
        catch(ArithmeticException e){
            System.err.print("\nException handled!");
        }
    }

    public static void main(String[] args){
        System.out.print("\f");
```

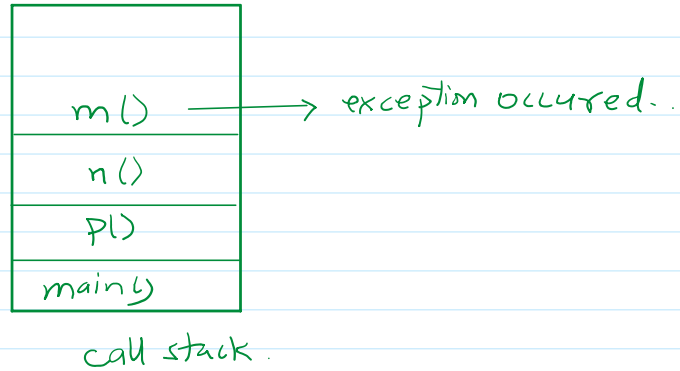
This is the example of Unchecked Exception.

In this example, an exception occur in m() method (due to division by zero), which is not handled there. The exception is then propagated to the n() method, and from there, it moves to the p() method, where it is finally caught and handled. After, the exception is dealt with the message "normal flow...." is printed indicating that the program continued without termination.

```
        ExceptionPropagation obj = new ExceptionPropagation();
        obj.p(); //   Starts the chain of method call.
        // The line is executed only if the exception is handled properly.
        System.out.print("\nNormal flow...");
    }
}
```

Output:



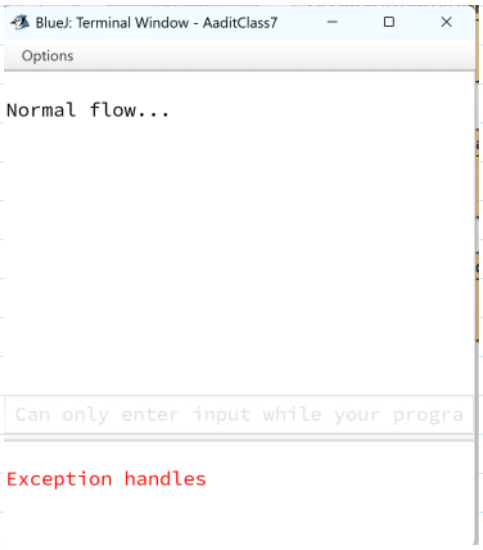


Note: By default, checked Exceptions are not forward in calling chain (propagated).

```
package ExceptionHandling;
import java.io.*;


/**
 * Write a description of class UnCheckedExceptionPropagation here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class UnCheckedExceptionPropagation
{
    // Method m throws a checked exception, so it must declare it with throws.
    public void m() throws IOException{
        // Explicitly throwing a checked exception.
        throw new IOException("Device error");
    }

    // Method n(), calls m() so it must also declare the same exception.
    public void n()throws IOException{
        m(); //   No handling here, so the exception is passed up the chain.
    }

    //  Method P () catches the exception using a try-catch block.
```

```
    public void p(){
        try
        {
            n(); //  Exception propagates here and is caught.
        }
        catch (IOException ioe)
        {
            System.err.print("\nException handles");
        }
    }

    public static void main(String[] args){
        UnCheckedExceptionPropagation ob = new UnCheckedExceptionPropagation();
        System.out.print("\f");
        ob.p(); //  Triggers the method chain.
        // Will be printed only if an exception is properly handled.
        System.out.print("\nNormal flow...");
    }
}
```

Output:



The code demonstrates how a checked exception (IOException) propagates through method calls. The m() method throws the exception which is propagated to n() and then to p(). In p() The exception is caught and handled, allowing the program to continue with normal execution.

What is throws keyword in Java?

The throws keyword is a keyword that is used to declare possible exceptions that a method can generate and allow those exceptions to be handled by the method that invokes it, rather than within the method itself.

Syntax:

```
Return_type method_name() throws exception_class_name{
…
}
Or
Return_type method_name() throws exception_class_name1,exception_class_name2…
exception_class_nameN{
…
}
```

Homework:

1. ATM Transaction System
Write a Java program to simulate an ATM machine. The program should:
- Allow users to enter PIN, withdraw, deposit, and check balance
- Throw exceptions for:
  - Invalid PIN (after 3 attempts)
  - Withdrawal amount exceeding balance
  - Negative transaction amounts
- Use custom exception InvalidPINException
- Implement try-catch-finally and ensure proper messages

2. Student Registration System
Create a Java program for student registration:
- Accept student name, age, and course
- Handle exceptions when:
  - Age is less than 16 or greater than 60
  - Name is null or empty
  - Invalid course selection
- Use a custom exception InvalidStudentDataException
- Demonstrate multiple catch blocks

3. Password Validation System
Write a Java program that validates a password:
- Conditions:
  - Minimum 8 characters
  - Must include uppercase, lowercase, digit, and special character
- Throw custom exception WeakPasswordException if invalid
- Use throw keyword and handle it properly

4. Electricity Bill Calculator
Develop a Java program to calculate electricity bill:
- Input units consumed
- Handle exceptions when:
  - Units are negative
  - Input is not numeric
- Use InputMismatchException and IllegalArgumentException
- Ensure program continues after handling exception

5. Online Exam System
Create a Java program that simulates an online exam:
- Accept answers for 5 questions
- Handle exceptions:
  - Invalid option input (only A/B/C/D allowed)
  - Null answers
- Use nested try-catch blocks
- Display score even if some answers are invalid

6. Matrix Operations Program
Write a Java program to:
- Accept two matrices and perform addition
- Handle exceptions:
  - Invalid matrix size mismatch
  - Array index errors

    - Non-integer input
- Use multiple catch blocks and proper error handling

7. Employee Attendance System
Develop a program that:
- Records employee login hours
- Throws exception if:
    - Hours exceed 24
    - Negative hours entered
    - Invalid employee ID
- Create custom exception InvalidAttendanceException
- Use throw and throws

8. Number Conversion Utility
Write a Java program that:
- Converts a string into integer, double, and boolean
- Handle:
    - NumberFormatException
    - Null values
- Use separate methods with throws keyword
- Handle all exceptions in main()

9. Voting Eligibility System
Create a Java program:
- Accept user age and nationality
- Allow voting only if:
    - Age $\geq$ 18
    - Nationality is valid
- Throw custom exception InvalidVoterException
- Use proper exception messages and handling

10. Travel Booking System
Write a Java program for booking tickets:
- Input passenger name, number of tickets, and destination
- Handle exceptions:
    - Invalid ticket count (negative or zero)
    - Null passenger name
    - Invalid destination
- Use custom exception InvalidBookingException
- Demonstrate use of finally block for confirmation message